

Breeding Planner

Engineering Handoff Report

Generated from docs/handoff Markdown sources in the repository.

Generated on 2026-04-26

Included sections:

- Engineering Handoff
- Architecture And Stack
- Product And Workflow Spec Audit
- Backend API And Data Model
- Testing And Quality Audit
- Decisions, History, And Risks

This PDF is a distribution copy of the repo handoff report for onboarding a new engineer into the Breeding Planner codebase.

Engineering Handoff

Engineering Handoff

This handoff documents the repository as it exists on 2026-04-25. It is an audit of the current codebase, not a rewrite of product intent. Where older docs disagree with the code, treat the code as authoritative.

Executive Summary

Breeding Planner is a hybrid codebase with two distinct architectural centers:

- A large, local-first breeder application centered on `src/App.jsx` (`../src/App.jsx`)
- A newer, more modular lab and shared-backend system centered on `src/features/lab` (`../src/features/lab`), `src/shared` (`../src/shared`), and `server/src` (`../server/src`)

The project currently ships in four runtime shapes:

- Browser app via Vite
- Electron desktop app
- Capacitor mobile wrappers for Android and iOS
- Express + Prisma shared backend for auth, lab catalog, pricing, orders, and result workflows

The most important transition fact is that the repository is mid-migration. Core breeder planning is still mostly local-state driven. Shared lab/auth flows are backend-driven. The result is a real hybrid system, not a cleanly unified architecture.

Start Here

For a new developer, this is the shortest reliable path into the codebase:

1. Read `architecture.md` (`./architecture.md`)
2. Read `product-spec.md` (`./product-spec.md`)
3. Read `backend-api-and-data.md` (`./backend-api-and-data.md`)
4. Read `testing-and-quality.md` (`./testing-and-quality.md`)
5. Read `decisions-and-risks.md` (`./decisions-and-risks.md`)

Quick Local Setup

Frontend:

```
npm install
npm run dev
```

Backend:

```
cd server
npm install
copy .env.example .env
npm run prisma:generate
npm run prisma:migrate
npm run prisma:seed
npm run dev
```

Default local URLs:

- Frontend: `http://127.0.0.1:5173`
- Backend: `http://127.0.0.1:4000`
- Backend health: `http://127.0.0.1:4000/api/health`

Frontend shared-backend configuration is controlled by VITE_API_URL.

Authoritative Code Paths

These files matter most when reading the system:

- src/index.jsx (../src/index.jsx): frontend entry point
- src/AuthShell.jsx (../src/AuthShell.jsx): top-level shell, auth gate, appearance, and hash-based surface switching
- src/App.jsx (../src/App.jsx): main breeder app, very large and state-heavy
- src/features/lab/LabAppShell.jsx (../src/features/lab/LabAppShell.jsx): lab portal shell and routing
- src/features/lab/api/client.ts (../src/features/lab/api/client.ts): compatibility layer between local handlers and shared backend
- src/shared/apiClient.ts (../src/shared/apiClient.ts): backend auth/session client
- server/src/app.ts (../server/src/app.ts): backend app composition
- server/prisma/schema.prisma (../server/prisma/schema.prisma): shared backend data model
- server/prisma/seed.ts (../server/prisma/seed.ts): seed users, catalog, pricing

Important Non-Authoritative or Legacy Areas

These exist in the repository but should not be treated as the primary implementation without confirming usage first:

- src/App.js (../src/App.js)
- src/App_prev.jsx (../src/App_prev.jsx)
- src/HEAD_App.jsx (../src/HEAD_App.jsx)
- src/main_App.jsx (../src/main_App.jsx)
- packages/core (../packages/core)
- apps/desktop (../apps/desktop)

They may contain useful history, but the active browser app currently routes through App.jsx, not those files.

Current Transition Risks

The main transition risks are:

- One very large monolithic breeder component (App.jsx)
- Mixed JavaScript and TypeScript with incomplete type coverage
- Multiple persistence modes in the same product
- Partial feature parity between local lab handlers and shared backend APIs
- Stale docs and old environment variable names still present in the repo
- No browser E2E coverage and no CI workflow that enforces test execution

These are expanded in decisions-and-risks.md (./decisions-and-risks.md).

Recommended First-Week Plan For A New Developer

1. Stand up both frontend and backend locally.
2. Confirm login, breeder view, lab portal, catalog edit, order creation, result submission, and certificate generation manually.
3. Trace data flow through AuthShell.jsx, App.jsx, LabAppShell.jsx, and src/features/lab/api/client.ts.
4. Read the backend Prisma schema and services before changing shared lab behavior.
5. Avoid broad refactors until the persistence split and active code paths are fully understood.

Architecture And Stack

Architecture And Stack

Stack Summary

Frontend:

- React 18
- Vite 5
- JavaScript + TypeScript mix
- Vitest
- Tailwind CSS 4
- i18next
- jsPDF / PDF tooling
- better-sqlite3 for local lab persistence in Node/Electron contexts

Desktop and mobile packaging:

- Electron 30
- Capacitor for Android and iOS wrappers

Backend:

- Node.js
- Express
- TypeScript
- Prisma
- PostgreSQL
- Zod
- JWT auth with refresh tokens

Supporting libraries used by product features include html5-qrcode, xlsx, and reactflow.

Runtime Surfaces

Surface	Entry point	Purpose	Notes
Web app	src/index.jsx (../src/index.jsx)	Main browser runtime	Vite dev/build target
Desktop app	electron/main.js (../electron/main.js)	Electron shell around the web UI	Thin shell with IPC persistence bridge
Mobile wrappers	capacitor.config.ts (../capacitor.config.ts)	Android/iOS packaging	Uses production web build
Shared backend	server/src/server.ts (../server/src/server.ts)	Auth, catalog, pricing, orders, results	Postgres + Prisma

Repo Map

Path	Purpose	Status
src (../src)	Frontend application code	Active
server (../server)	Shared backend	Active
electron (../electron)	Desktop shell	Active

android (../android) / ios (../ios)	Capacitor platform projects	Present
public (../public)	Static assets	Active
tests (../tests)	Domain-level tests	Active
packages/core (../packages/core)	Legacy/shared package experiment	Likely stale
apps/desktop (../apps/desktop)	Secondary app scaffolding	Likely stale

Frontend Composition

Top-level flow:

1. src/index.jsx renders AuthShell
2. AuthShell.jsx applies appearance and shared-backend providers
3. AuthShell.jsx switches surfaces using window.location.hash
4. The hash determines whether the user sees:
 - the breeder app via App.jsx
 - the lab app via src/features/lab/LabAppShell.jsx

This project does not currently use React Router. Routing is custom and hash-based.

Main Architectural Split

1. Breeder Core

The breeder surface is still dominated by src/App.jsx (../src/App.jsx), which contains a large amount of application state and behavior in one file. It covers:

- snake records
- pairings and planner flows
- genetics views and helpers
- rack/space management
- QR and label workflows
- backup/export/import behavior
- breeder-side lab integrations

This is the largest technical concentration point in the repository.

2. Lab And Shared Backend

The lab surface is relatively more modular:

- UI and routes in src/features/lab (../src/features/lab)
- service logic in src/services/lab (../src/services/lab)
- backend access in src/features/lab/api/client.ts (../src/features/lab/api/client.ts)
- auth/session logic in src/shared (../src/shared)
- backend domain logic in server/src/services (../server/src/services)

This area is where most recent architectural investment has gone.

Persistence Modes

One of the most important things to understand in this codebase is that data does not live in one place.

Data area	Primary storage	Notes
Core breeder planner data	Browser localStorage	Used heavily by App.jsx

Desktop local persistence	Electron IPC to local app data	Supplements browser storage in Electron mode
Local lab store	better-sqlite3 or in-memory/browser fallback	Managed by src/db/labStore.ts (../src/db/labStore.ts)
Shared lab/auth/order data	PostgreSQL through Express/Prisma backend	Required for multi-device workflows
UI/backend status flags	localStorage	Shared-backend session and status are cached locally

This means the app is not simply "offline" or "online". It is hybrid.

Shared Backend Model

Shared backend features currently include:

- authentication
- role-based access (breeder, lab, admin)
- test catalog
- pricing configuration
- shed-test order creation
- order status transitions
- result draft/submit flow
- certificate issuance support

The frontend is designed to work against either local handlers or the backend for some lab features. That bridge lives in src/features/lab/api/client.ts (../src/features/lab/api/client.ts).

Known Structural Constraints

Monolith In App.jsx

App.jsx is the dominant complexity hotspot. It is large, state-heavy, and not fully covered by TypeScript because the root tsconfig.json does not typecheck .jsx files.

Mixed Language Boundaries

The repository uses:

- .jsx and .js in large frontend areas
- .ts and .tsx in newer frontend modules
- strict TypeScript on the backend

This means safety and maintainability vary by subsystem.

Dual Code Paths

Lab functionality is split across:

- local service handlers
- shared backend API paths

Feature parity is not complete. Some operations explicitly remain unavailable on the shared backend path.

Active Versus Legacy Files

Active primary files:

- src/App.jsx
- src/AuthShell.jsx
- src/features/lab/LabAppShell.jsx

- src/shared/apiClient.ts
- server/src/

Files that appear to be historical or secondary:

- src/App.js
- src/App_prev.jsx
- src/HEAD_App.jsx
- src/main_App.jsx

These should be treated as context, not source of truth.

Architecture Recommendations For Transition

Short-term:

- keep changes localized to the active code paths
- document whether a change is local-only, backend-only, or both
- test both breeder and lab surfaces when touching shared flows

Medium-term:

- reduce App.jsx by extracting clear domain modules
- converge local/shared data paths where possible
- add explicit routing and feature ownership boundaries
- increase TypeScript coverage on the active frontend surface

Product And Workflow Spec Audit

Product And Workflow Spec Audit

What This Document Is

This is a spec audit derived from the codebase. There is no separate formal product-spec repository in this project. The actual product behavior currently lives in the code.

Product Scope

Breeding Planner combines breeder planning tools with a genetics-testing lab workflow.

At a high level the product includes:

- breeder animal and pairing management
- genetics planning and outcome estimation
- rack/space management
- QR and label workflows
- breeder-side lab order submission
- lab-side catalog, pricing, intake, result entry, and certificate workflows
- multi-role authentication for shared features

Roles

Role	Capabilities
breeder	Uses breeder planner, submits lab orders, views own results and certificates
lab	Works in the lab portal, processes orders, enters and submits results
admin	Full lab/admin surface, including catalog and pricing administration

Public sign-up is breeder-focused. Lab/admin access is seeded/admin-managed rather than a public self-service flow.

Breeder-Side Feature Inventory

The breeder surface in App.jsx includes these main functional areas:

- animal record management
- morph/genetics entry and display
- pair planning and breeding workflows
- project goals and suggestions
- rack, tub, and enclosure layout tracking
- labels and printable assets
- QR-related lookups and flows
- backups/export/import
- breeder profile information
- breeder-side lab order and result views

This is a broad product surface and much of it shares one component tree.

Genetics And Advisor Features

Domain-heavy logic exists around:

- Punnett-style genetics calculations in `src/genetics/punnett.ts` (`../../src/genetics/punnett.ts`)
- goal matching/scoring in `src/goals/goal.ts` (`../../src/goals/goal.ts`)
- suggestion generation and flowchart logic in `src/features/suggestions/api.ts` (`../../src/features/suggestions/api.ts`)

Important caveat:

- search-provider and LLM scaffolding exist
- not every integration is fully wired for production by default
- some advisor features depend on environment configuration or remain defensive/no-op without it

Lab Portal Feature Inventory

The lab portal is reachable through hash-based routes in `src/features/lab/LabAppShell.jsx` (`../../src/features/lab/LabAppShell.jsx`).

Current route groups include:

- dashboard
- incoming orders
- sample intake
- result entry
- completed tests
- admin oversight
- test catalog
- pricing logic
- order details
- developer tools

The presence of developer tools in the live shell is itself a current maintenance concern.

Shared Lab Workflow

Typical intended shared workflow:

1. Breeder signs in
2. Breeder creates a shed-test order
3. Order is stored on the shared backend
4. Lab users review incoming work
5. Lab enters result drafts and submits final results
6. Completed results update breeder-visible data and certificates
7. Breeder can view completed outcomes on their side

This workflow is actively implemented but still sits on top of a compatibility layer that supports both local and shared paths.

Certificate Workflow

Current certificate behavior is tied to completed lab results. The recent implementation direction in the repository is:

- result submission finalizes the order
- certificate data is generated from order/test/snake/breeder data
- certificate output is available on the breeder side

The certificate template and PDF generation live in:

- `src/services/lab/certificateTemplate.ts` (`../../src/services/lab/certificateTemplate.ts`)

- src/utils/pdf/labCertificatePdf.ts (../../src/utils/pdf/labCertificatePdf.ts)
- src/services/lab/certificateService.ts (../../src/services/lab/certificateService.ts)

Routing Model

The product uses custom hash routing instead of a formal router package.

Implications:

- route state is lightweight
- deep linking is possible only within the custom hash model
- route ownership is less explicit than in a conventional router setup
- navigation logic can be harder to audit because it is spread through UI state and hash parsing

Persistence And User Expectations

The product does not have one universal persistence rule.

Breeder users may experience:

- local-only behavior for planner data
- backend-backed behavior for shared auth and lab flows
- Electron-specific persistence behavior on desktop

This matters for onboarding a new engineer because bugs can come from the boundary between those modes, not only from the screen where the bug is visible.

Current Functional Constraints

The codebase itself shows several constraints that should be treated as part of the current product spec:

- the shared backend is required for reliable multi-device lab/auth behavior
- local and shared implementations do not yet have full feature parity
- some admin/lab actions are intentionally blocked on the shared path
- product behavior is partly defined by compatibility logic rather than a single canonical service boundary

Implicit Spec Sources

The most useful spec sources in the repo are:

- UI text and behavior in src/App.jsx and src/features/lab/*/
- domain services in src/services/lab/*/
- tests under src/*/test., tests//, and server/src/tests/*/
- Prisma schema and backend service logic

There is no strong separation today between "spec", "application service", and "UI behavior". A new developer should expect to reconstruct product rules from multiple layers.

Backend API And Data Model

Backend API And Data Model

Backend Purpose

The backend exists to support shared, multi-device parts of the product. It is not the source of truth for the entire breeder planner yet.

Current backend responsibilities:

- authentication
- role-based access control
- lab test catalog
- pricing configuration
- shared order lifecycle
- result submission and retrieval

Primary files:

- server/src/app.ts (../server/src/app.ts)
- server/src/server.ts (../server/src/server.ts)
- server/src/routes (../server/src/routes)
- server/src/services (../server/src/services)
- server/prisma/schema.prisma (../server/prisma/schema.prisma)

Environment Variables

Backend environment variables documented in server/.env.example (../server/.env.example):

Variable	Required	Notes
DATABASE_URL	Yes	PostgreSQL connection string
JWT_SECRET	Yes	Access/refresh token signing secret
CORS_ORIGIN	No	Allowed origins
PORT	No	Defaults to 4000
NODE_ENV	No	development or production

Frontend environment variables that matter to backend connectivity:

Variable	Notes
VITE_API_URL	Shared backend base URL used by the frontend

Route Inventory

Auth Routes

Defined in server/src/routes/authRoutes.ts (../server/src/routes/authRoutes.ts):

Method	Path	Purpose
POST	/api/auth/register	Create breeder account
POST	/api/auth/login	Sign in

POST	/api/auth/recover-password	In-app recovery flow
POST	/api/auth/refresh	Refresh expired access token
GET	/api/auth/me	Return current user

Lab Config Routes

Defined in server/src/routes/labRoutes.ts (../../server/src/routes/labRoutes.ts):

Method	Path	Purpose
GET	/api/lab/tests/catalog	Fetch lab test catalog
GET	/api/lab/tests/pricing	Fetch pricing configuration
PATCH	/api/lab/tests/catalog/:id	Edit catalog item
PATCH	/api/lab/pricing/:id	Edit pricing rule

Order Routes

Defined in server/src/routes/orderRoutes.ts (../../server/src/routes/orderRoutes.ts):

Method	Path	Purpose
POST	/api/lab/orders/calculate-price	Server-side price calculation
POST	/api/lab/orders	Create order
GET	/api/lab/orders	List orders
DELETE	/api/lab/orders	Bulk delete orders
DELETE	/api/lab/orders/:id	Delete one order
GET	/api/lab/orders/:id	Get order details
POST	/api/lab/orders/:id/results/draft	Save draft result
POST	/api/lab/orders/:id/results/submit	Submit final result
PATCH	/api/lab/orders/:id/status	Update order status

Prisma Data Model

Current main Prisma models:

Model	Purpose
User	Auth and role model
ShedTestCatalog	Test definitions and catalog metadata
PricingConfig	Pricing logic configuration
ShedTestOrder	Shared order header
ShedTestOrderAnimal	Animals attached to an order
ShedTestOrderAnimalTest	Tests ordered per animal
ShedTestOrderResult	Saved/submitted result records

Seed data is defined in server/prisma/seed.ts (../../server/prisma/seed.ts).

Backend Service Layout

The backend is relatively cleanly separated by service:

- auth logic in auth services and route handlers
- catalog/pricing logic in lab configuration services
- order creation and mutation logic in order services

Controllers are generally thin. Service files are the real behavior layer.

Shared Backend Boundaries

Important architectural reality:

- the backend is authoritative for shared lab and auth behavior
- it is not authoritative for all breeder planner state
- the frontend still contains compatibility logic for local-mode behavior

This means backend changes should be reviewed in the context of `src/features/lab/api/client.ts`, not in isolation.

Known Gaps And Caveats

The frontend compatibility client explicitly shows that some operations are not fully supported on the shared backend path yet. Examples include:

- creating new catalog tests from the shared path
- some pending shed/batch-style operations
- certain download/admin utilities
- some payment-status related actions

For a new developer, this is a warning sign: not every feature visible in local lab services or old UI affordances is guaranteed to exist end-to-end on the backend.

Migration History

The Prisma migrations indicate an actively evolving backend. The current state includes multiple recent migrations for:

- refresh-token auth support
- lab order numbering
- catalog metadata expansion

Treat schema history as active project history, not a stable long-settled layer.

Testing And Quality Audit

Testing And Quality Audit

Test Strategy Today

The repository has meaningful unit and domain tests, but coverage is uneven and there is no end-to-end test layer.

Broadly:

- core genetics and planner logic have unit/domain tests
- some lab services have targeted tests
- shared auth client logic has targeted tests
- backend auth and selected service logic have tests
- UI integration coverage is thin
- browser E2E coverage is absent

Frontend And Shared Test Inventory

Frontend and root-level tests currently include:

Area	Example files	What it covers
Main app utility behavior	src/App.test.js (../src/App.test.js)	Selected helpers extracted from App.jsx
Lab utility layout logic	src/features/lab/utis/labelLayout.test.js (../src/features/lab/utis/labelLayout.test.js), src/features/lab/utis/labelSizing.test.js (../src/features/lab/utis/labelSizing.test.js)	Label dimensions and layout
Lab certificate logic	src/services/lab/certificateTemplate.test.ts (../src/services/lab/certificateTemplate.test.ts)	Certificate field mapping
Lab genetics update logic	src/services/lab/geneticsUpdateEngine.test.ts (../src/services/lab/geneticsUpdateEngine.test.ts)	Snake genetics mutation rules from lab results
Numbering helpers	src/services/lab/orderNumber.test.ts (../src/services/lab/orderNumber.test.ts), src/services/lab/testNumber.test.ts (../src/services/lab/testNumber.test.ts)	Human-readable order/test number formatting
Pricing	src/services/pricing/calculateLabOrderPrice.test.js (../src/services/pricing/calculateLabOrderPrice.test.js), tests/labPricing.test.ts (../tests/labPricing.test.ts)	Lab price calculations
Shared API config/client	src/shared/config/api.test.js (../src/shared/config/api.test.js), src/shared/apiClient.test.js (../src/shared/apiClient.test.js)	API URL rules, auth refresh handling
Genetics and goals	src/genetics/punnett.test.ts (../src/genetics/punnett.test.ts), tests/genetics.test.ts (../tests/genetics.test.ts), tests/goals.test.ts (../tests/goals.test.ts), tests/score.test.ts (../tests/score.test.ts)	Domain engine coverage

Planner utilities	tests/planner.test.ts (../tests/planner.test.ts), tests/labelPresets.test.ts (../tests/labelPresets.test.ts), tests/qrLookupInput.test.ts (../tests/qrLookupInput.test.ts)	Planner-adjacent logic
-------------------	---	------------------------

Backend Test Inventory

Backend tests currently include:

File	Coverage
server/src/tests/auth.test.ts (../server/src/tests/auth.test.ts)	Register/login/recovery/auth flows
server/src/tests/labConfigService.test.ts (../server/src/tests/labConfigService.test.ts)	Catalog/pricing service behavior
server/src/tests/orderNumber.test.ts (../server/src/tests/orderNumber.test.ts)	Backend order numbering

What Is Covered Well Enough To Be Useful

The current tests are most useful in these areas:

- deterministic genetics calculations
- lab numbering helpers
- certificate field mapping
- genetics mutation rules after lab results
- shared auth refresh behavior
- selected backend service validations

These tests are valuable because they protect high-risk logic without depending on a browser.

Major Gaps

No Browser E2E Coverage

There are no Playwright/Cypress/Selenium style end-to-end tests in the repository.

Implication:

- login, breeder workflows, lab workflows, and certificate UI behavior are primarily verified manually

Limited UI Integration Tests

Most screen-level behavior is not exercised through rendered component tests. This is especially important because much of the breeder product lives in one large component.

Partial Backend Test Depth

The backend has tests, but they do not comprehensively cover every route/service combination, role check, or order lifecycle edge case.

Type Safety Does Not Cover The Main Monolith

The root TypeScript configuration does not typecheck .jsx files. That leaves the main active file, src/App.jsx, outside strict type checking.

CI And Release Automation

Current GitHub Actions appear focused on build/deploy packaging rather than test enforcement:

- GitHub Pages deployment workflow
- Windows installer build workflow

What is missing:

- required frontend test workflow
- required backend test workflow
- required lint/typecheck workflow
- browser E2E workflow

This means passing CI is not the same thing as the repository being well tested.

Useful Local Verification Commands

Frontend:

```
npm run build
npm test
```

Backend:

```
cd server
npm run build
npm test
```

Targeted examples:

```
npm test -- src/shared/apiClient.test.js
npm test -- src/services/lab/geneticsUpdateEngine.test.ts
cd server && npm test -- src/tests/auth.test.ts
```

Quality Recommendations

Immediate:

- add a CI workflow that runs frontend build, backend build, frontend tests, and backend tests on every PR
- add smoke tests for login, order creation, result submission, and certificate rendering

Next:

- add browser E2E coverage for breeder and lab critical paths
- add component-level tests around the most fragile breeder UI flows
- expand backend route coverage beyond auth and selected service slices

Longer-term:

- move more active frontend logic into typed modules that can be tested in isolation

Decisions, History, And Risks

Decisions, History, And Risks

Important Note

Some of the decisions below are inferred from code structure, commit history, and migration patterns rather than being explicitly documented by the original author. They are still useful because they explain why the repository looks the way it does today.

Inferred Decision History

1. Local-First Origins

The breeder planner appears to have started as a local-first application:

- heavy use of browser storage
- Electron persistence bridge
- large all-in-one frontend logic

This explains why a lot of business logic still lives in the frontend rather than the backend.

2. Packaging Came Before Full Backend Centralization

Electron and Capacitor support exist, suggesting the product prioritized "run everywhere" packaging before fully converging on a single shared backend model.

3. Shared Backend Was Added Later For Lab And Multi-User Needs

The newer backend, auth, and lab modules show a more modular architecture than the legacy breeder surface. This strongly suggests the shared backend was introduced to support:

- multi-device access
- roles and authentication
- lab operations
- centralized order and result workflows

4. Compatibility Was Chosen Over A Hard Rewrite

Instead of replacing all local behavior at once, the codebase introduced compatibility layers, especially around lab APIs. This reduced immediate rewrite cost, but it also created overlapping execution paths.

High-Priority Risks

Large Monolithic Frontend Surface

src/App.jsx is the single biggest maintainability risk in the codebase. It carries too much state and too many responsibilities.

Impact:

- harder onboarding
- harder refactoring
- weaker test isolation
- higher regression risk for seemingly small UI changes

Split Persistence Model

The app uses local storage, Electron persistence, local lab database behavior, and shared

backend state at the same time.

Impact:

- bugs can come from synchronization boundaries
- ownership of data can be unclear
- migration work is harder because there is no single truth source

Partial Shared-Backend Parity

Not every lab or admin behavior has a complete shared-backend implementation.

Impact:

- UI affordances can mislead developers into assuming a server path exists
- features can behave differently depending on runtime mode

Mixed JS/TS With Incomplete Type Coverage

The backend is stricter than the frontend. The most critical frontend file is not strongly typechecked.

Impact:

- refactors in active UI code are riskier than they look
- cross-module contracts are weaker in the breeder surface

Documentation Drift

Some existing docs and environment examples no longer match the current code.

Examples:

- top-level README language count was outdated
- root .env.example contains old variable names
- legacy files exist beside active files with similar names

Impact:

- onboarding friction
- misleading setup steps
- wasted time debugging the wrong source

No E2E Safety Net

The product has broad user-facing workflows but no browser E2E coverage.

Impact:

- regressions are likely to be caught manually and late
- release confidence depends heavily on ad hoc testing

Codebase Smells Worth Calling Out

- developer tools are still present in the lab shell
- some temporary debug comments/logs remain in service code
- stale scaffolding directories remain in the repo
- the project contains multiple app files with overlapping names

None of these are catastrophic alone. Together they increase cognitive load.

Practical Guidance For The Next Developer

Do:

- confirm whether a bug belongs to local mode, shared mode, or both before editing
- read the active frontend entry files before assuming structure from older files
- validate both frontend and backend builds after touching shared flows
- use the tests as spec hints, not just regression guards

Do not:

- assume App.js is the live breeder implementation
- assume all lab actions exposed in UI are fully backend-supported
- assume existing docs are current unless they match the code
- start with a large architectural rewrite before mapping data ownership

Recommended Next Engineering Moves

Short-term:

- remove or isolate remaining dev-only lab tools
- document active environment variables in one place
- add CI that runs tests and builds on every PR

Medium-term:

- split App.jsx by domain ownership
- define a clear migration policy for local versus shared data
- add explicit route ownership and navigation structure

Long-term:

- converge on a single persistence strategy for shared-critical data
- move more business rules into typed domain services with strong tests
- add browser E2E coverage for the highest-value flows

Suggested Ownership Boundaries

If another developer is joining, the cleanest initial ownership split is probably:

- Developer A: breeder core and App.jsx decomposition
- Developer B: lab/shared backend and API parity

Trying to own both surfaces at once without separation will slow transition down.